CT421 Graph Colouring

Joey Galvin 22348861

Step 1

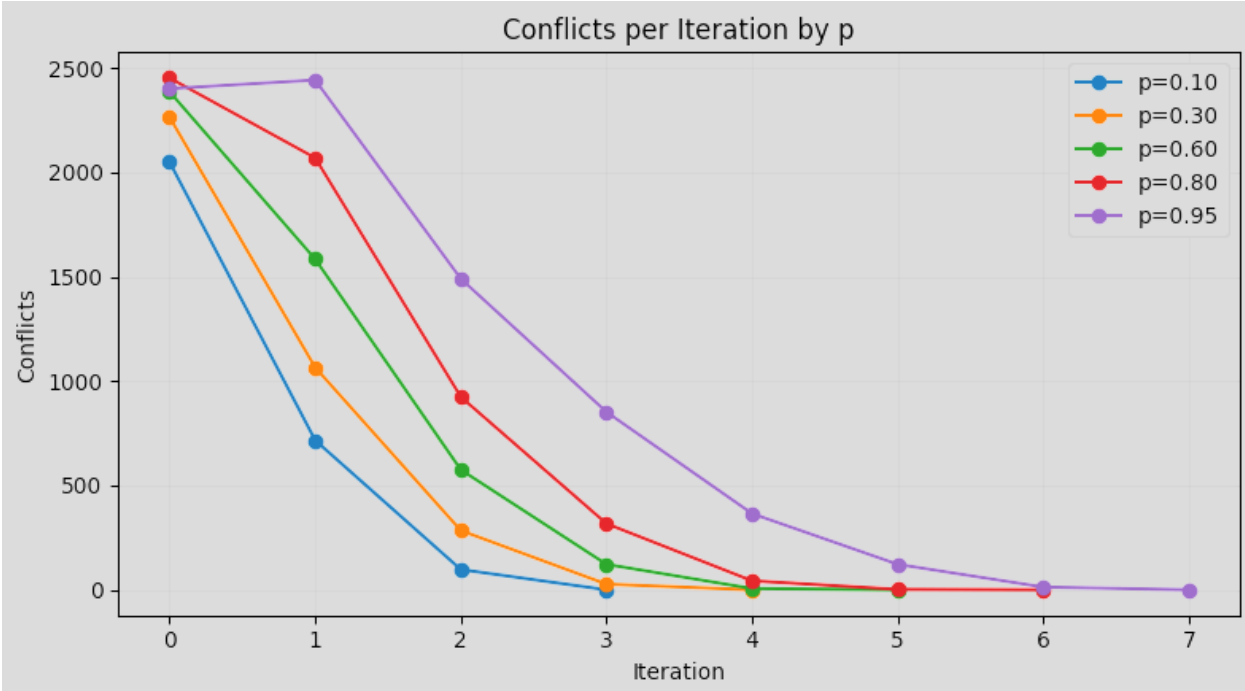
Graph Topology

The Erdos-Renyi graph model was the model I decided to use for this step. The graph is made using 2 simple parameters; n which is the number of nodes the graph will have and p which is a float between 0 and 1 which is the probability that any pair of nodes will share an edge.
The first reason I chose this model is its very simple to understand and code and still creates complex and customizable (dense or non-dense) graphs.I thought ring, lattice or bipartite graphs would have been too simple and not realistic to any real life graph colouring applications. Also P could be varied to study the convergence behaviour of different density graphs.

Algorithm

The algorithm I used was a synchronous local update rule. Each node in the graph only knows its colours of its neighbours and not the global state of the graph.
At each iteration of the algorithm, every node in the graph checks its current local state to see if it is conflicted with a neighbour. If it is in conflict it selects a new colour from the currently available palette. If all colours of the current palette are being used by a neighbour, it instead chooses a random new one. Every node being updated synchronously means I was able to easily graph the amount of conflicts changing over each iteration.
The number of colours available initially, k, was set to the highest degree node + 1 for each graph, so a denser graph will have a higher k. With a graph of 5000 nodes and a p=0.8 we would expect a node with about a max degree of about 4000 for example. This essentially guarantees a colour for each graph.

Results



The experiment was run across 5 different P values and on a graph with 5000 nodes.

```
P_VALUES := []float64{0.1, 0.3, 0.6, 0.8, 0.95}
```

As seen above, all runs successfully reached zero conflicts within 7 Iterations which shows that the algorithm I described above creates a valid colouring quite efficiently. The graph shows the denser the graph becomes, the longer it takes to converge as expected.

Step 2

Research Question

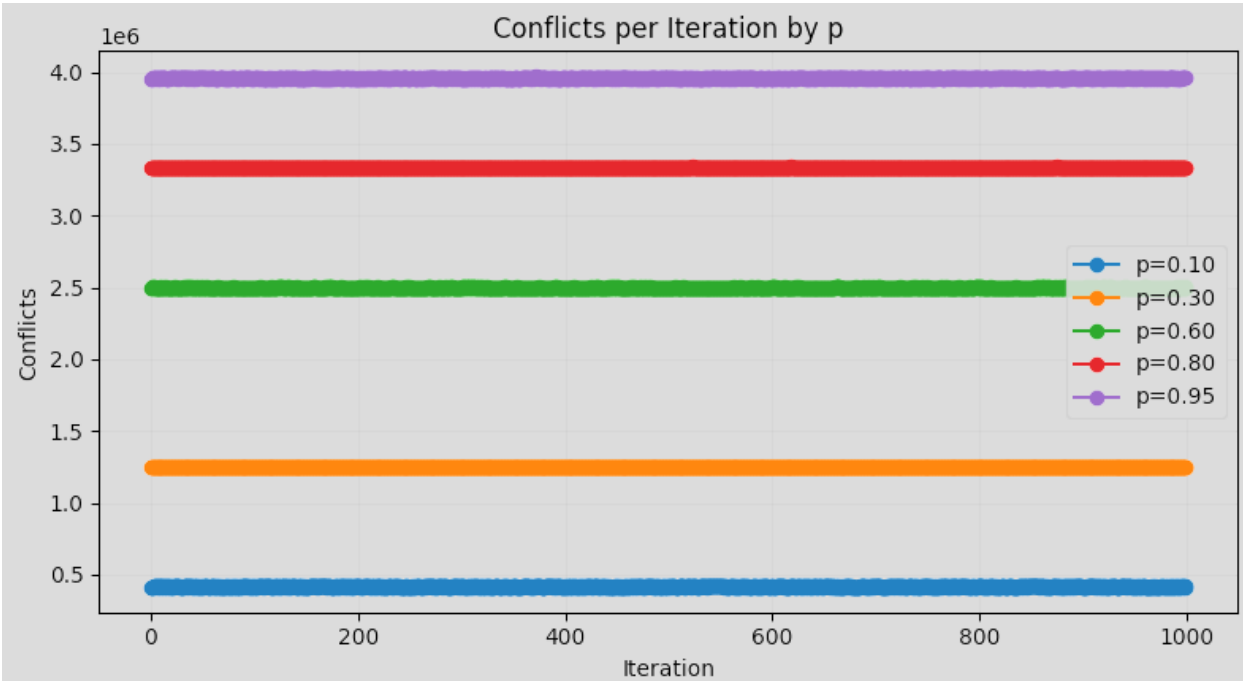
For step 2 I wanted to explore the effect of the changing k; the number of colours initially in the palette on convergence speed/ability to converge.

Approach

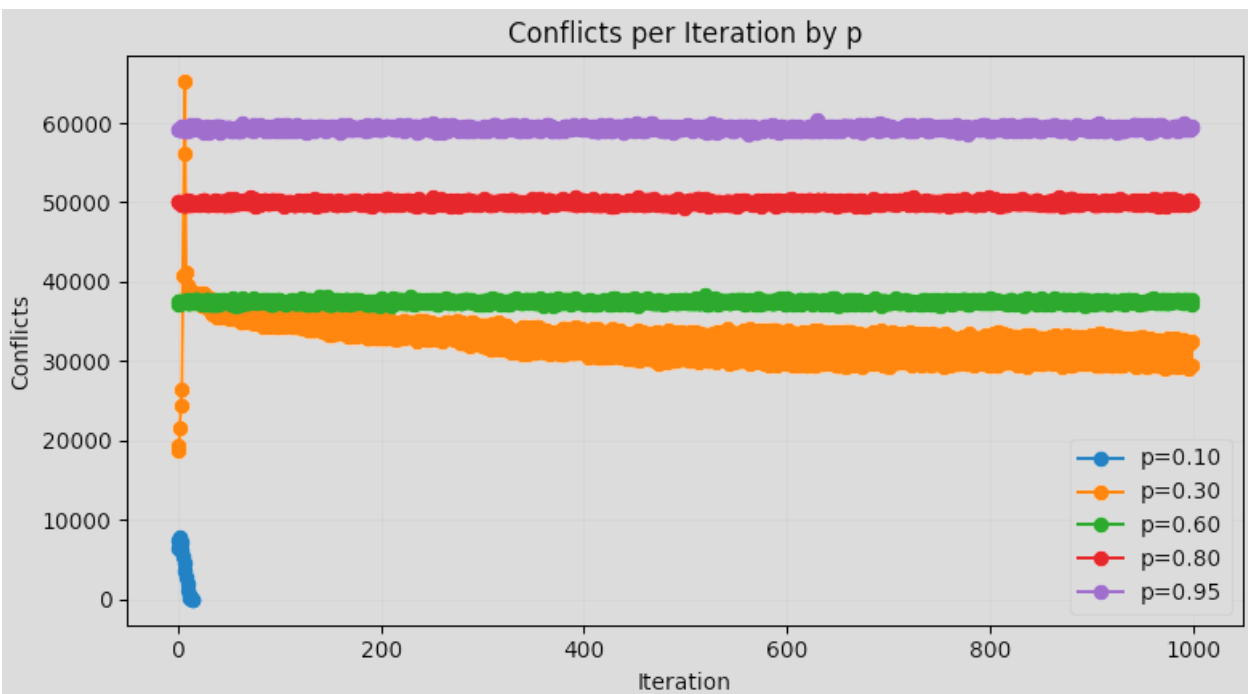
To study this, I'm going to keep the initial results which use k = max degree of all nodes + 1 and run my algorithm a couple of more times using different values of k and simply observe the resulting plots.

Results

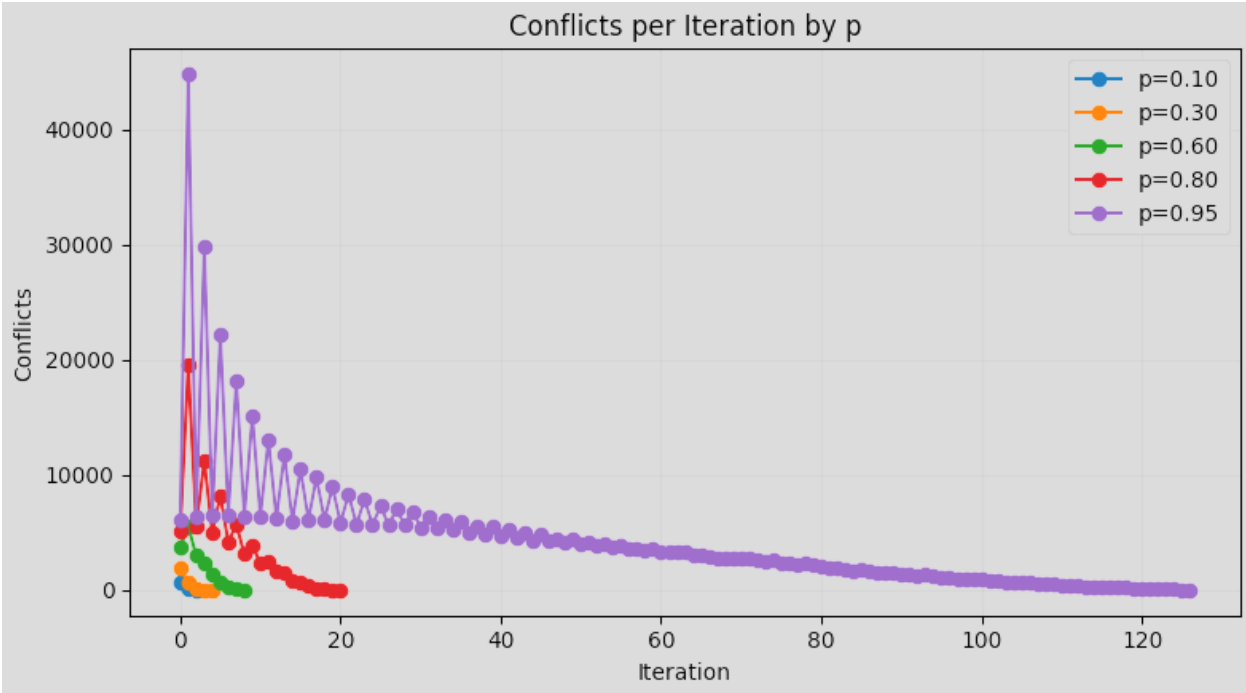
k=3



With k=3, the results were quite predictable. Since there were only 3 colours to split among 5000 nodes, it was quite obvious it was not going to converge and find a valid colouring in the defined number of maximum iterations (1000).



With k=200, we see a convergence, p=0.10 converges after 15 iterations while the rest still fail to converge after 1000 iterations although p=0.30 shows some movement.



Next I tried a much larger k=2000, every graph converged with the densest graph converging after 126 iterations and the least converging after just 2. However the most notable thing coming from this result is the 'bouncyness' of the results. This is due to the synchronised nature of the algorithm I described above, since every node updates at the same time. 2 conflicting neighbours will look at each other and both change at the same time instead of just one.

Conclusion

In this experiment I definitely proved how k, the number of colours available for the graphs scales dramatically the speed and success rate of convergence in graph colouring problems. Below certain thresholds, graphs consistently failed to find a valid colouring even after 1000 iterations.

I also think I proved that a distributed approach to graph colouring is a decent approach to finding a valid solution for a graph colouring problem, it may not find the optimal solution but will find you a good solution after a relatively small amount of time.